Programming in the Era of AI

Daniel Lemire, professor
Université du Québec (TÉLUQ)
Montréal 🇨🇦

blog: https://lemire.me

X: @lemire · GitHub: github.com/lemire

Where I am coming from

- Author of high-performance libraries integrated into major browsers, runtimes and standard libraries: **simdutf**, **fast_float**, **simdjson**, **Roaring Bitmaps**.
- Among the top 2% of scientists worldwide (Stanford/Elsevier), 100+ peer-reviewed papers.
- Among the 1000 most-followed developers on GitHub.
- **And: I have not typed most of my code by hand in over a year.**

Preface

Back in the 1990s

The singularity

> *« I believe that the creation of greater than human intelligence will occur during the next thirty years. (I'll be surprised if this event occurs before 2005 or after 2030.) »*
>
> Vernor Vinge, 1993

Part 1

Where we actually are



Daniel Lemire
@lemire

Boost

What fraction of your code is typed by hand (no AI)



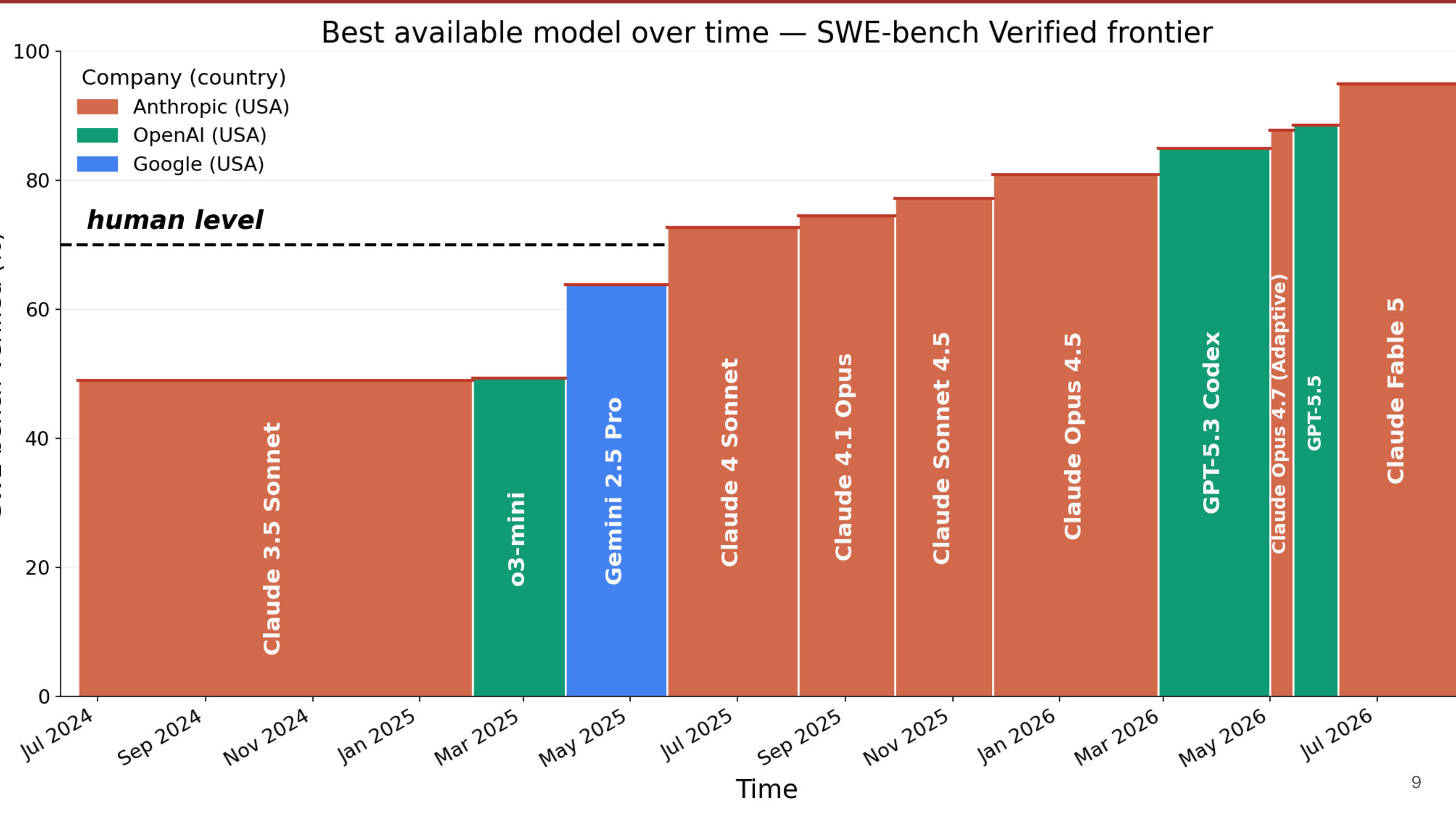
484 votes · Final results

8:59 PM · Oct 19, 2026 · **3,809** Views

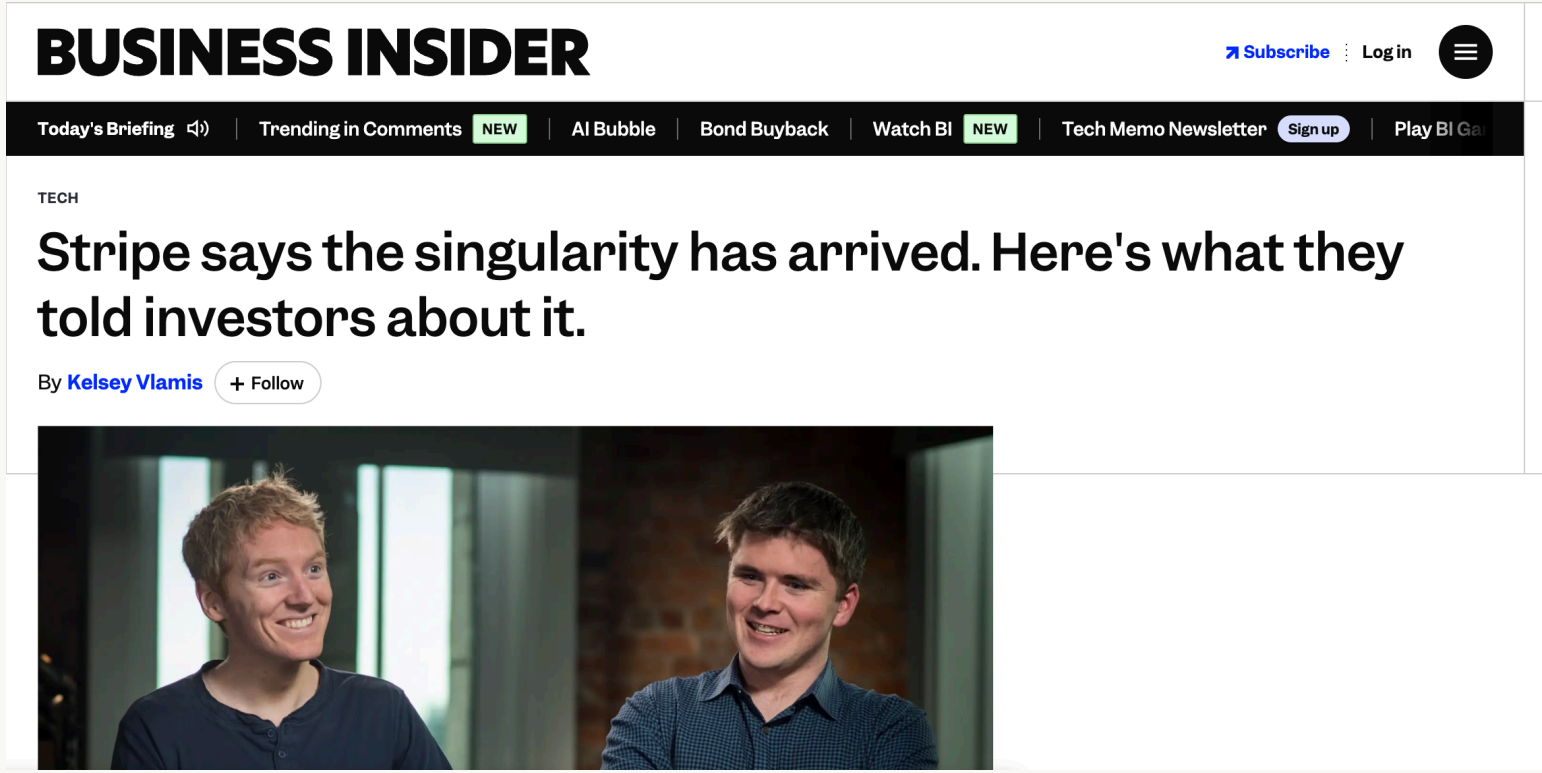
16 11 6

SWE-bench Verified

- Real GitHub issues from real open-source Python repositories.
- The model gets the repository, the issue text, and nothing else.
- Success = the project's own **hidden tests** pass afterwards.
- 500 tasks, human-validated as solvable.


Best available model over time — SWE-bench Verified frontier
Company (country)
Anthropic (USA)
OpenAI (USA)
Google (USA)
human level
100
80
60
40
20
0
Claude 3.5 Sonnet
o3-mini
Gemini 2.5 Pro
Claude 4 Sonnet
Claude 4.1 Opus
Claude Sonnet 4.5
Claude Opus 4.5
GPT-5.3 Codex
Claude Opus 4.7 (Adaptive)
GPT-5.5
Claude Fable 5
Jul 2024
Sep 2024
Nov 2024
Jan 2025
Mar 2025
May 2025
Jul 2025
Sep 2025
Nov 2025
Jan 2026
Mar 2026
May 2026
Jul 2026
Time

> *January 1st marked the beginning of the singularity.*
>
> —Stripe, August 19, 2026



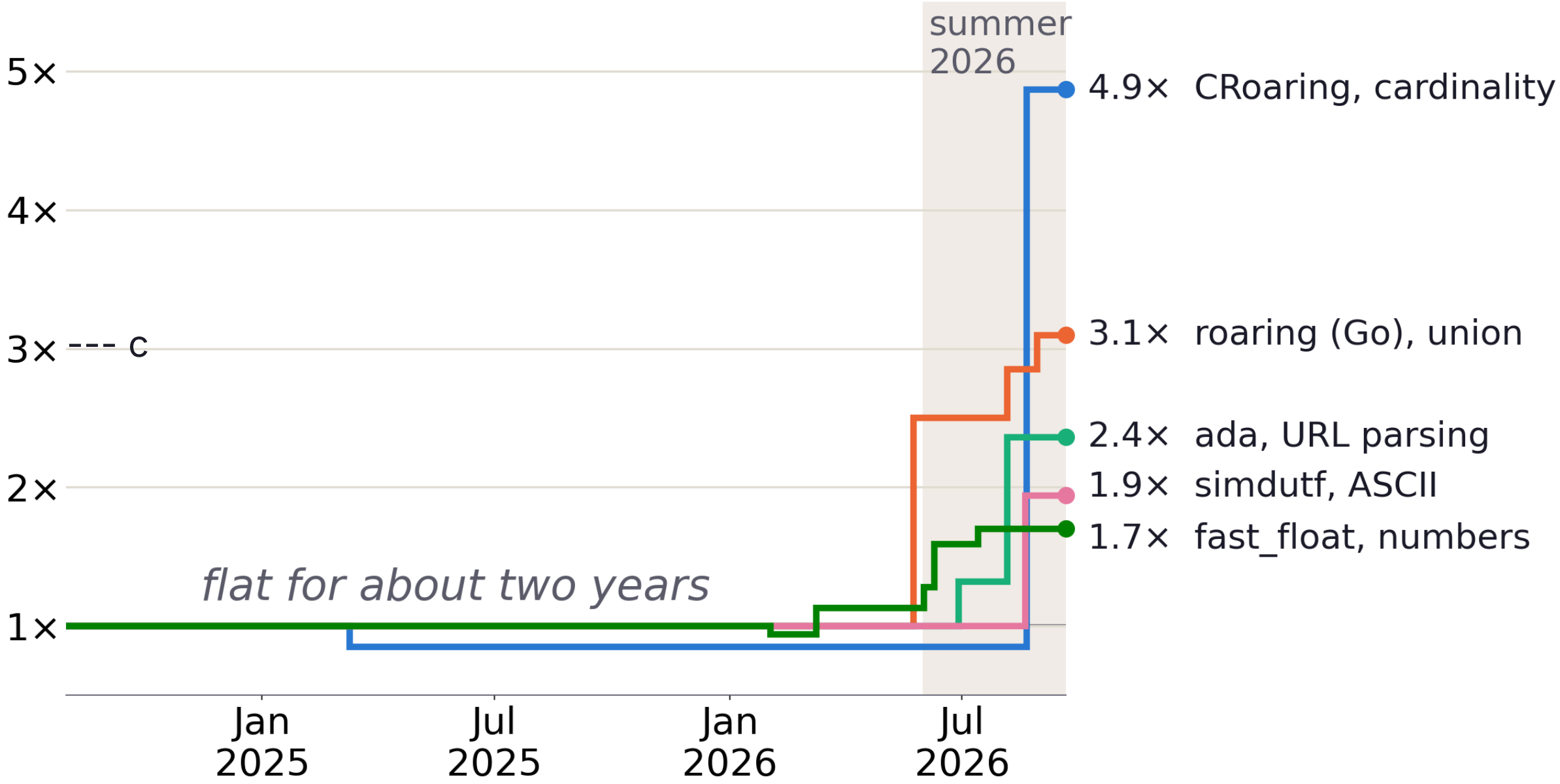
> *People often talk about this concept called AGI, meaning artificial general intelligence, that is, an AI as intelligent as a person. I actually think we crossed that threshold about three months ago.*
>
> — Marc Andreessen, May 19, 2026



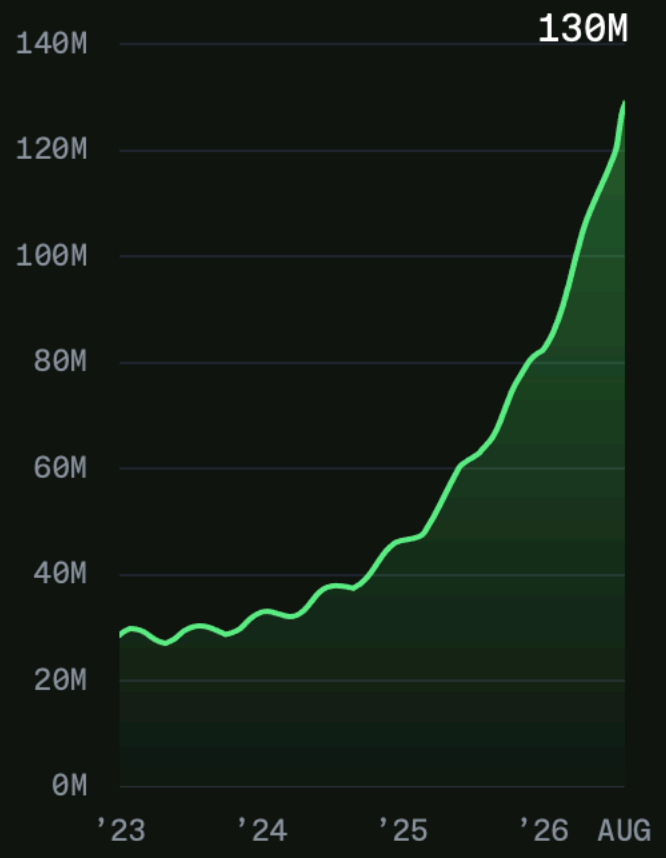
A summer of AI optimization

- Six mature open-source libraries: roaring, CRoaring, ada, fast_float, simdjson, simdutf.
- Shipped in GCC, Chromium, Node.js... used by millions.
  d

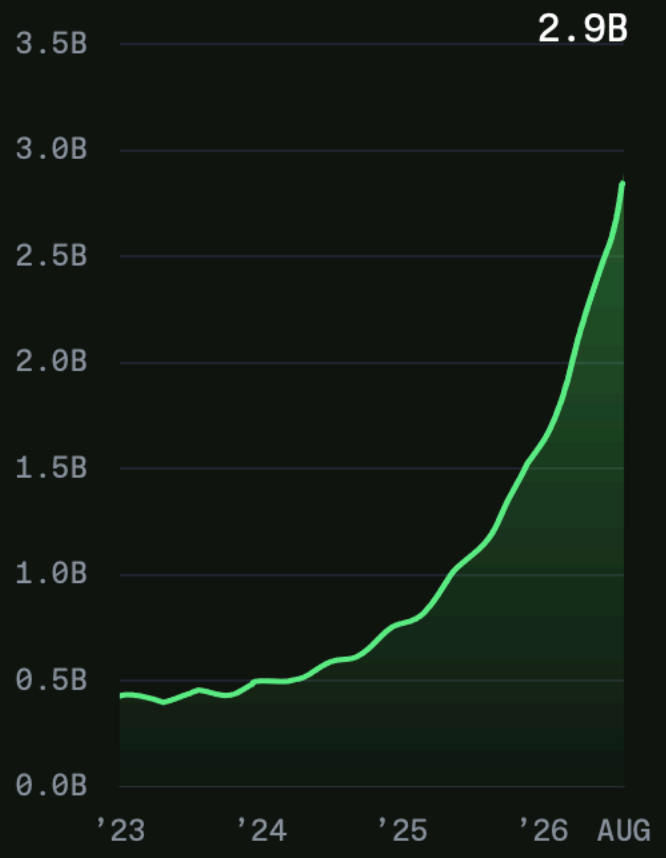
Speedup vs. August 2024, every commit, same machine



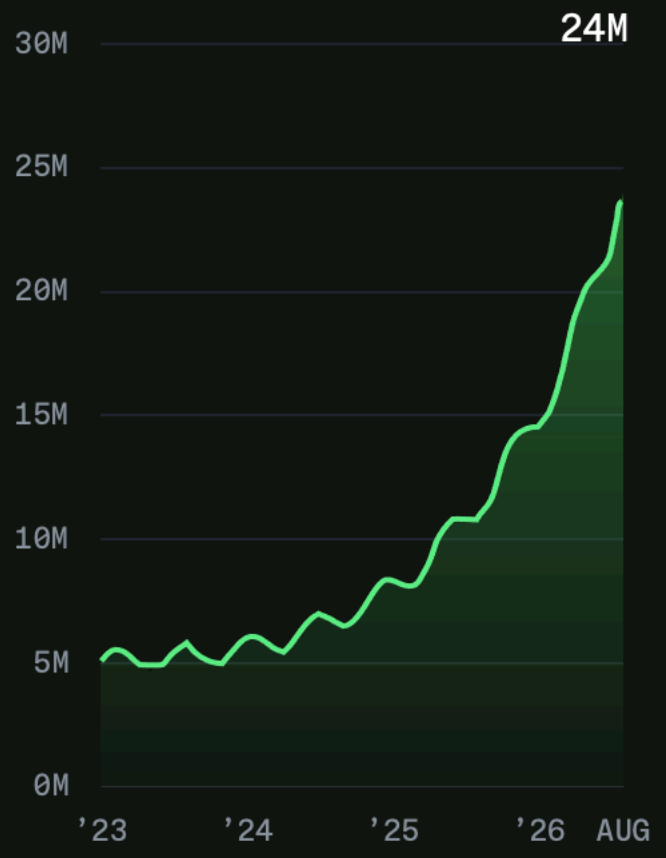
Merged pull requests per month



Commits per month



New repos per month



Classified as Microsoft Confidential

Jevons paradox

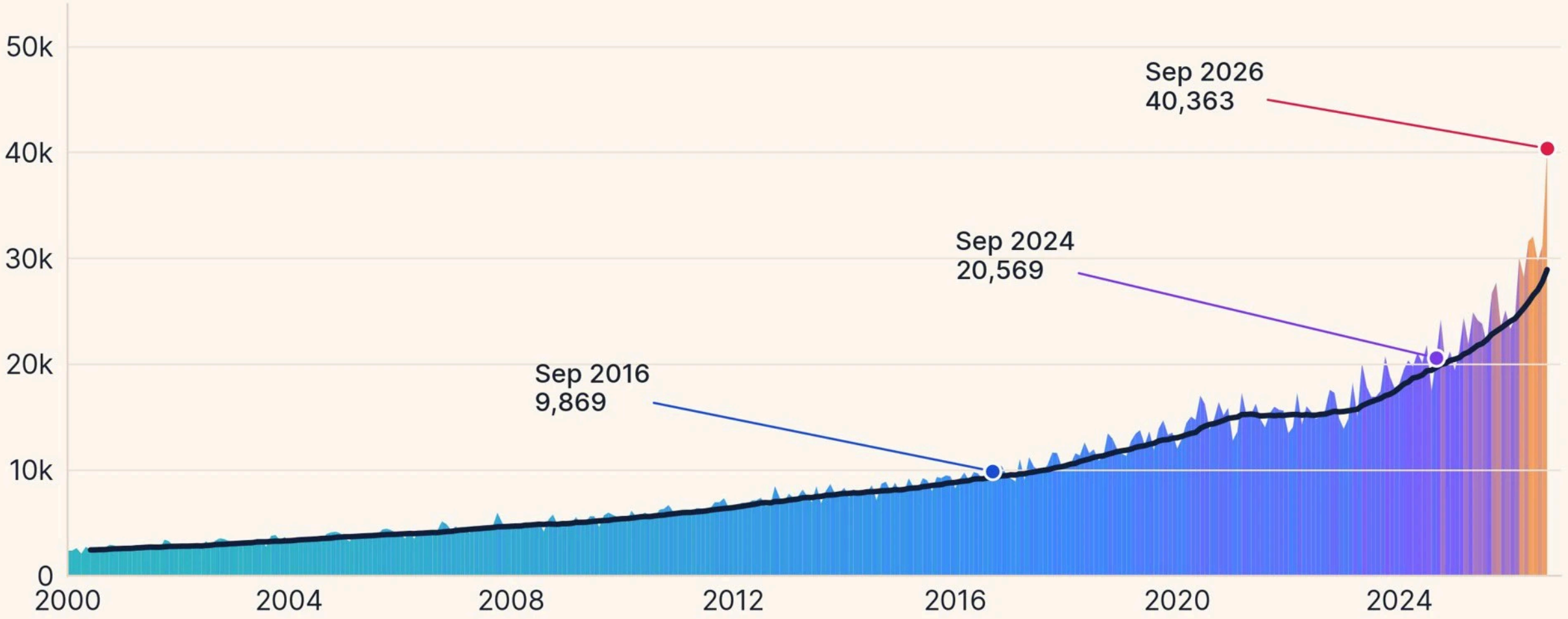
- 1865: William Stanley Jevons, *The Coal Question*.
- James Watt's steam engines burned far less coal per unit of work...
- ...and Britain's coal consumption **went up**, not down.

When something gets cheaper to use, we use much more of it.

- Tokens got cheaper: we consume far more tokens.
- Code got cheaper: we write far more code, and review, test, host and maintain it.

January 2000 to September 2026. October 2026 left out: the month is not finished.

Papers posted to arXiv each month



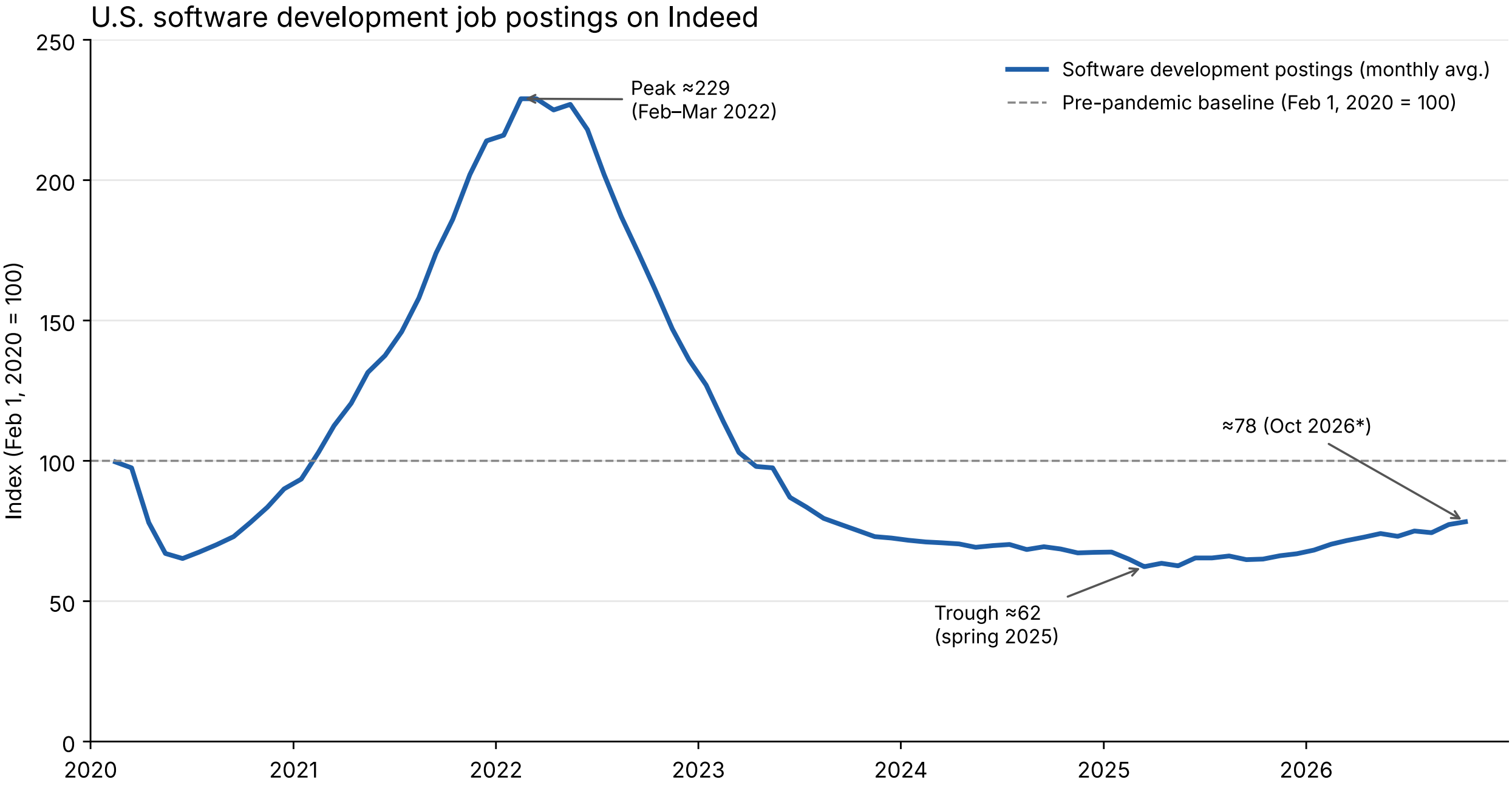
Source: arXiv monthly submissions. Color: the month. Dark line: 12-month average.

arXiv capped submissions at two a month

- October 1, 2026: **two submissions per person per month.**
- September 2026: over 40,000 papers, up from about 20,000 two years earlier.
- Doubling every two years. Moderated by volunteers.
- Google stopped accepting new bug reports in its open-source bounty program: they could not cope with the influx.

Producing got cheap. Human attention did not.

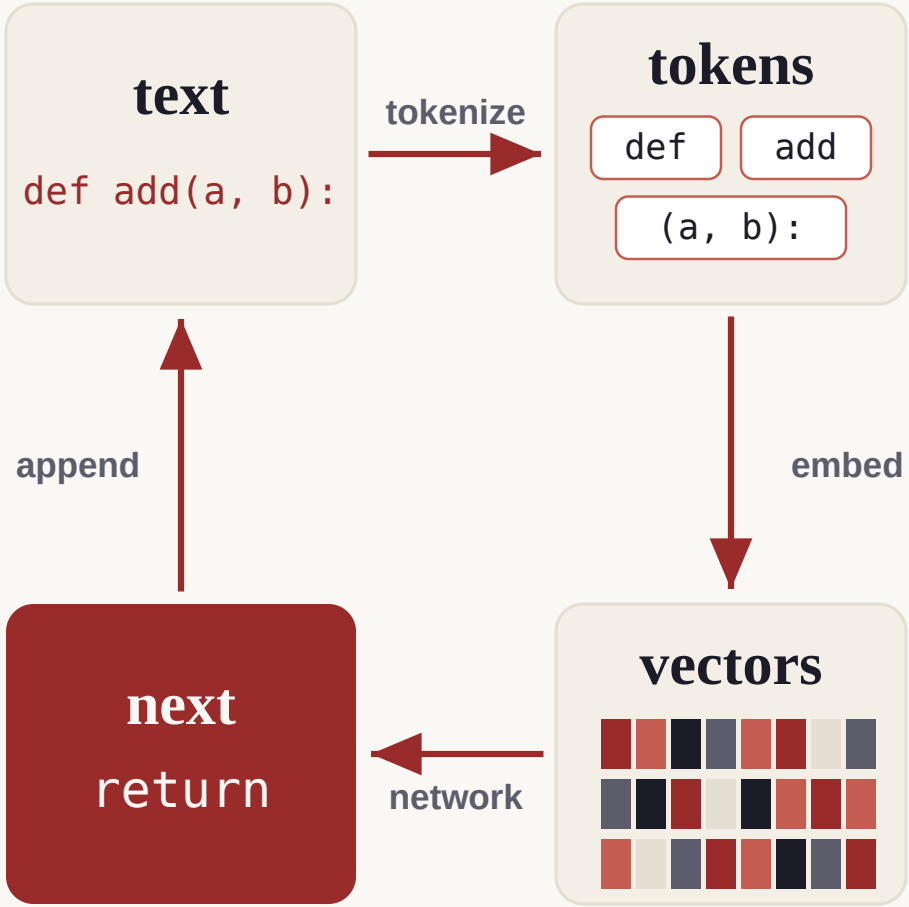
lemire.me/blog/2026/10/04/arxiv-capped-submissions-at-two-a-month/



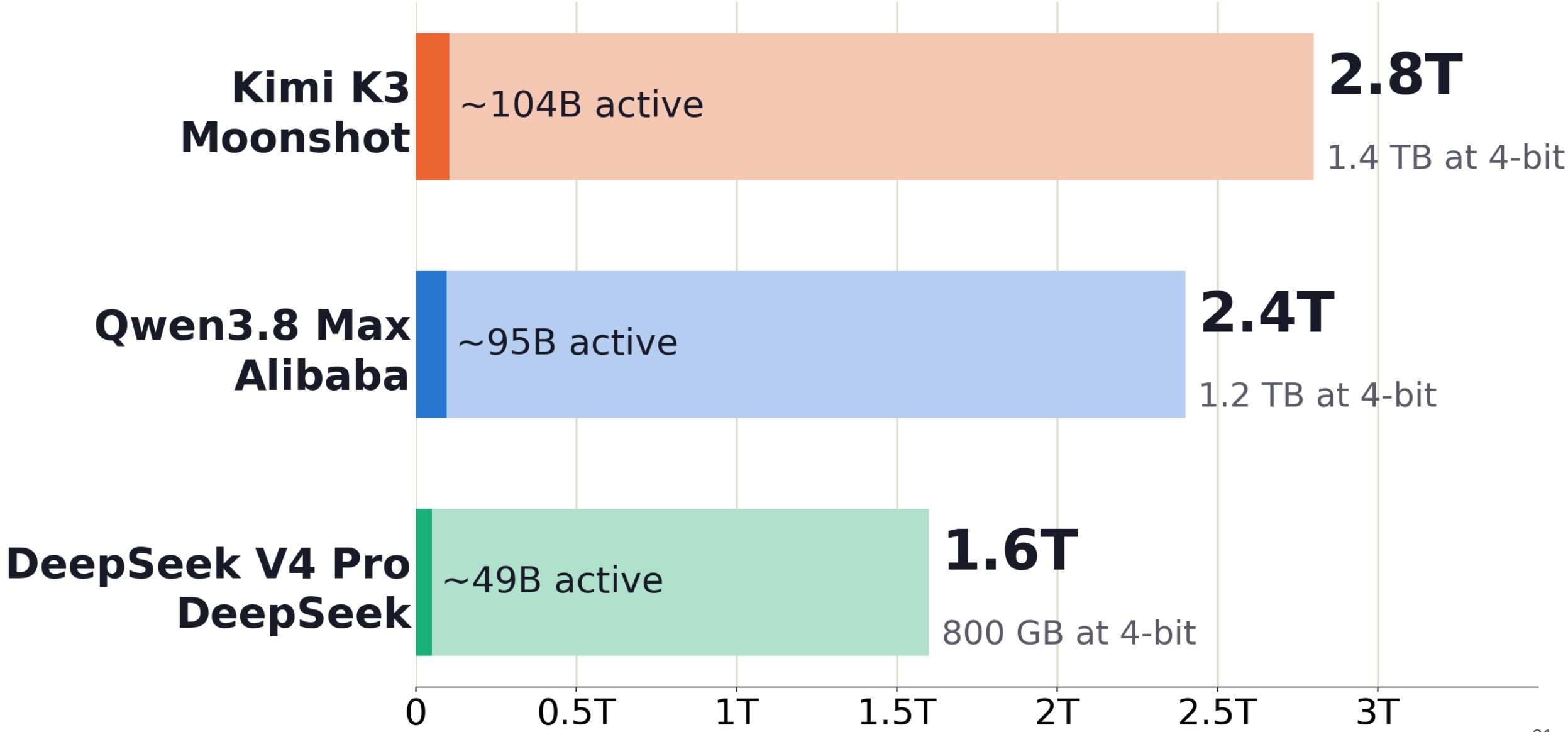
Source: Indeed Hiring Lab via FRED (IHLIDXUSTPSOFTDEVE), seasonally adjusted. Monthly averages of daily values; *Oct 2026 = Oct 1–16.

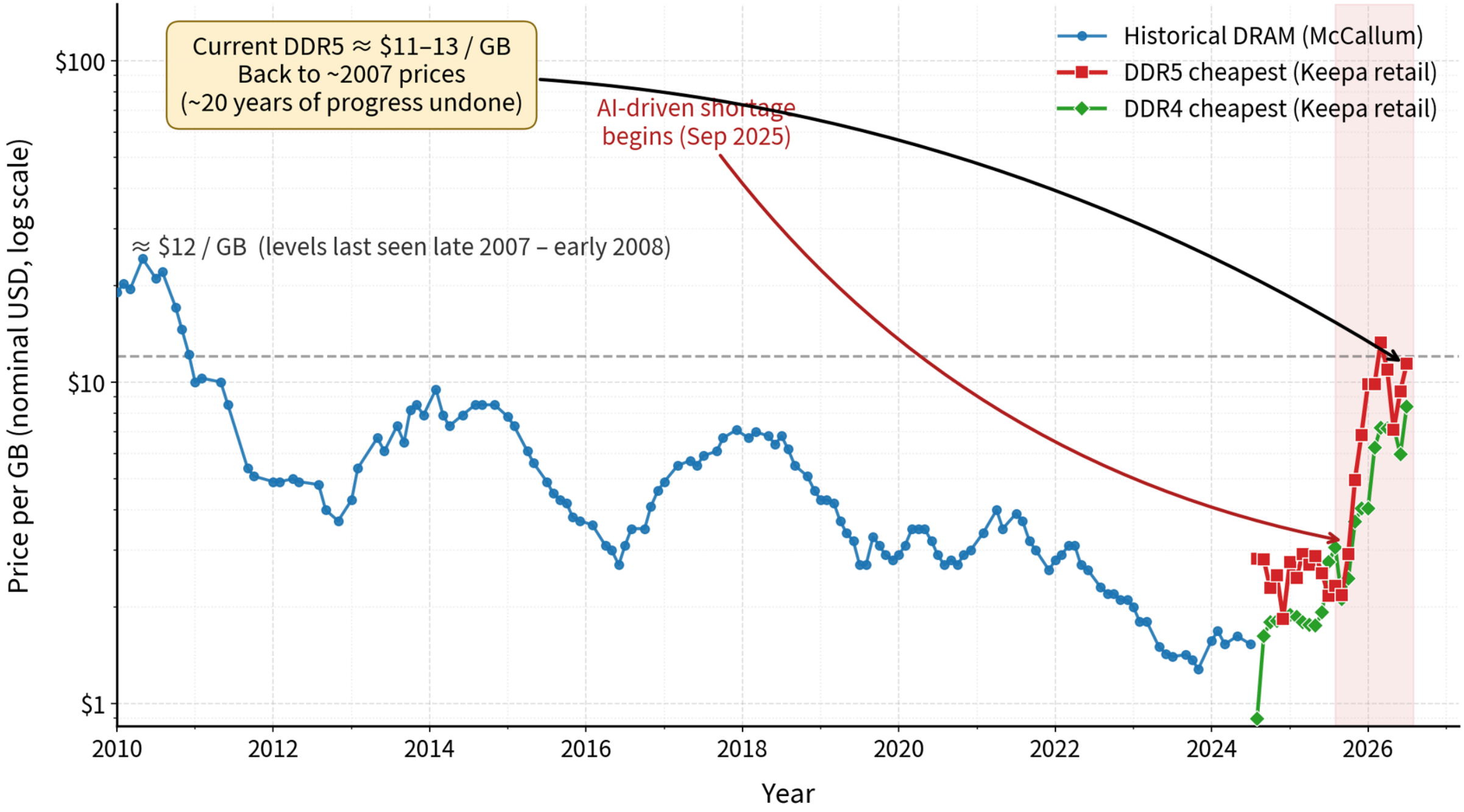
Memory

How an LLM works



Open-weight models: total parameters (dark = active per token)



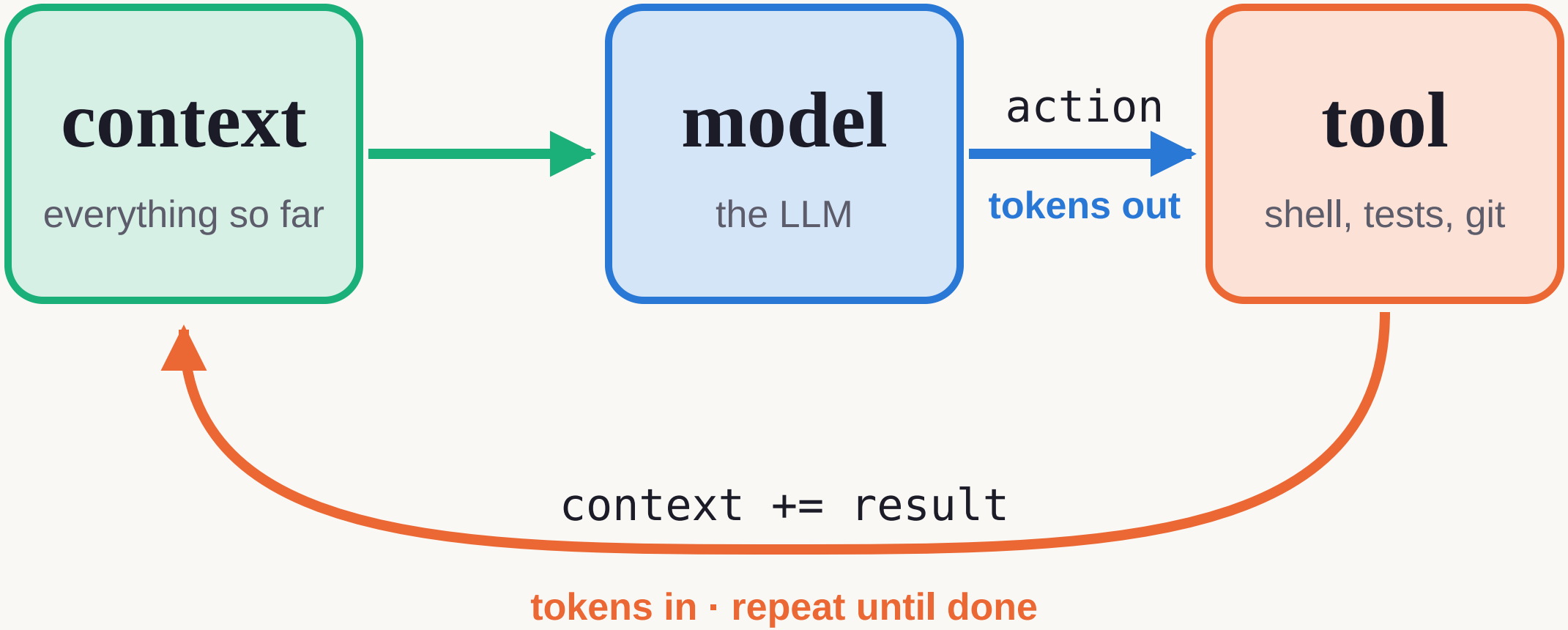


Sources: John C. McCallum historical dataset (via Stanford DAM) + Keepa Amazon retail prices for cheapest consumer modules • Nominal USD • Data through July 2026

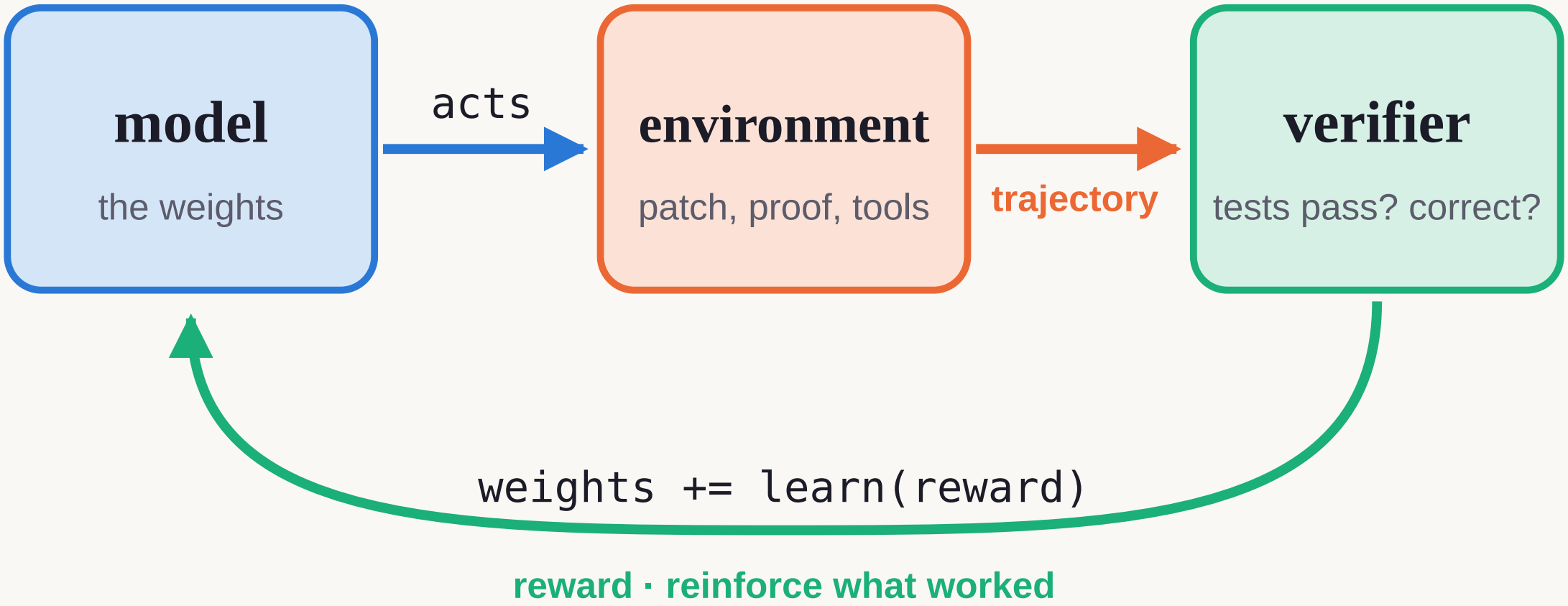
How can a language model write software?

- I write a query, it is maybe 80 characters.
- I get back a full program with 100,000 lines.
- Not how it works, there is more information invovled.

The inference loop



Reinforcement learning



Challenges



Daniel Lemire
@lemire

Boost

Someone found a bug in my code 3 days after I released, should I...

Issue a patch release now	28.1%
wait a week	8.5%
drink more beer	**63.4%**

481 votes · Final results

2:42 PM · Sep 5, 2026 · **5,884** Views

7 1 16 2

Teaching

- How to teach software development in the AI era. @fromdevoid
- “Is it still worth learning to program?” @birkenkrahe
- Are security risks a difference in degree or in kind? @gcomeaudotcom
- How to think, how to set goals, and how to express yourself clearly. @JoyfulBadger

Quality

- Include the tests! @nabendu82
- Can you verify the code? @MatRopert
- How good are the tests? @dyaroshev
- How to have rigor? @opdroid1234
- Someone still needs to explain/understand? @jrdi

Emerging ideas

Emerging idea: ephemeral testing

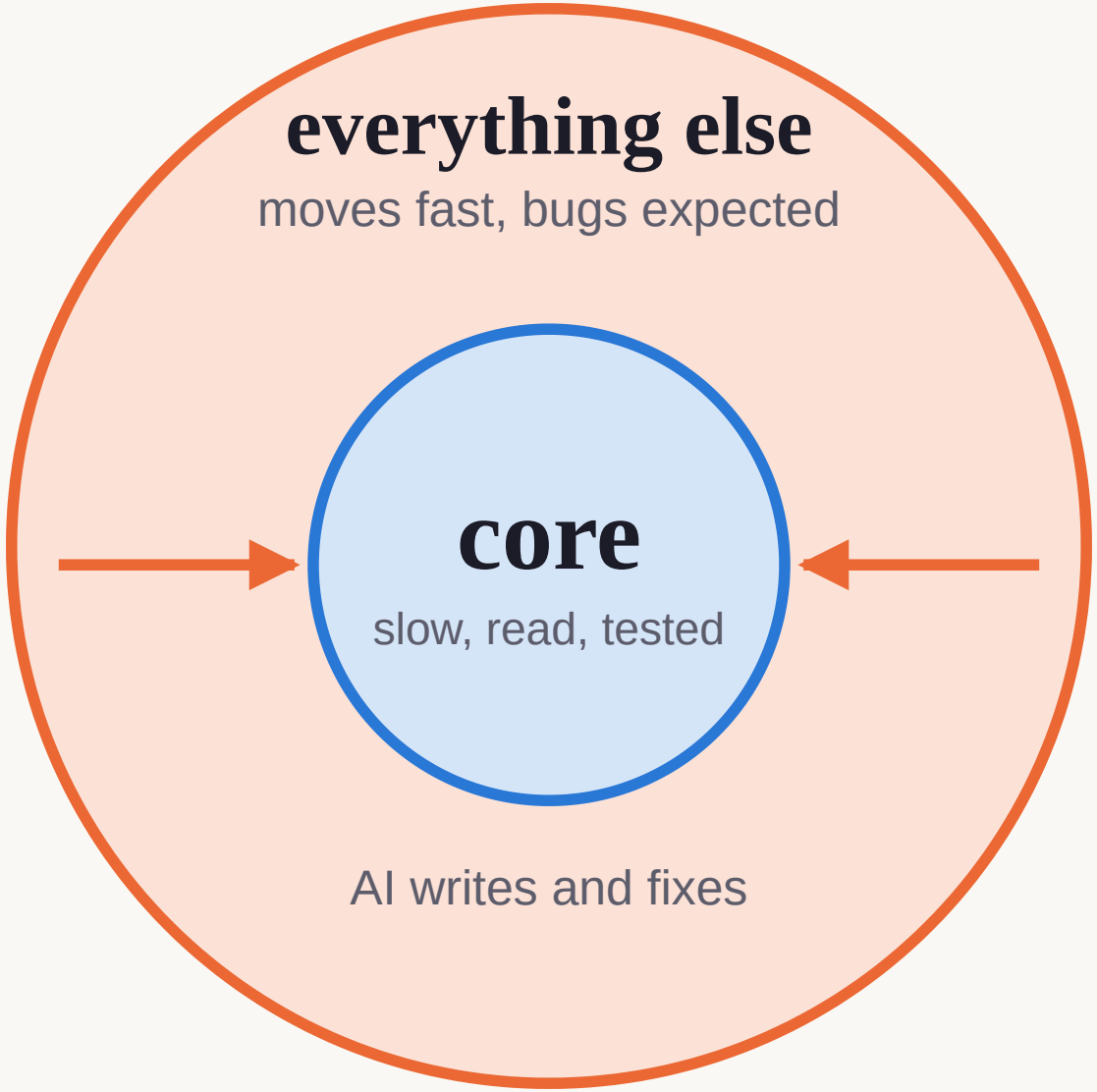
1. Build a component (by hand or with an agent).
2. Ask an agent to build **throwaway software on top of it**: an app, another layer.
3. Have it test what it built.
4. Judge the foundation by how well the layers above work. Then discard them.

lemire.me/blog/2026/10/05/ephemeral-testing/

Emerging idea: a layered model

- Keep a **small core** that changes slowly and on purpose.
- For the core, you read the code. You insist on tests. No vibe coding.
- **Everything else can move fast.** There will be bugs, but the AI fixes them quickly.
- **The core cannot depend on the outer layers.**

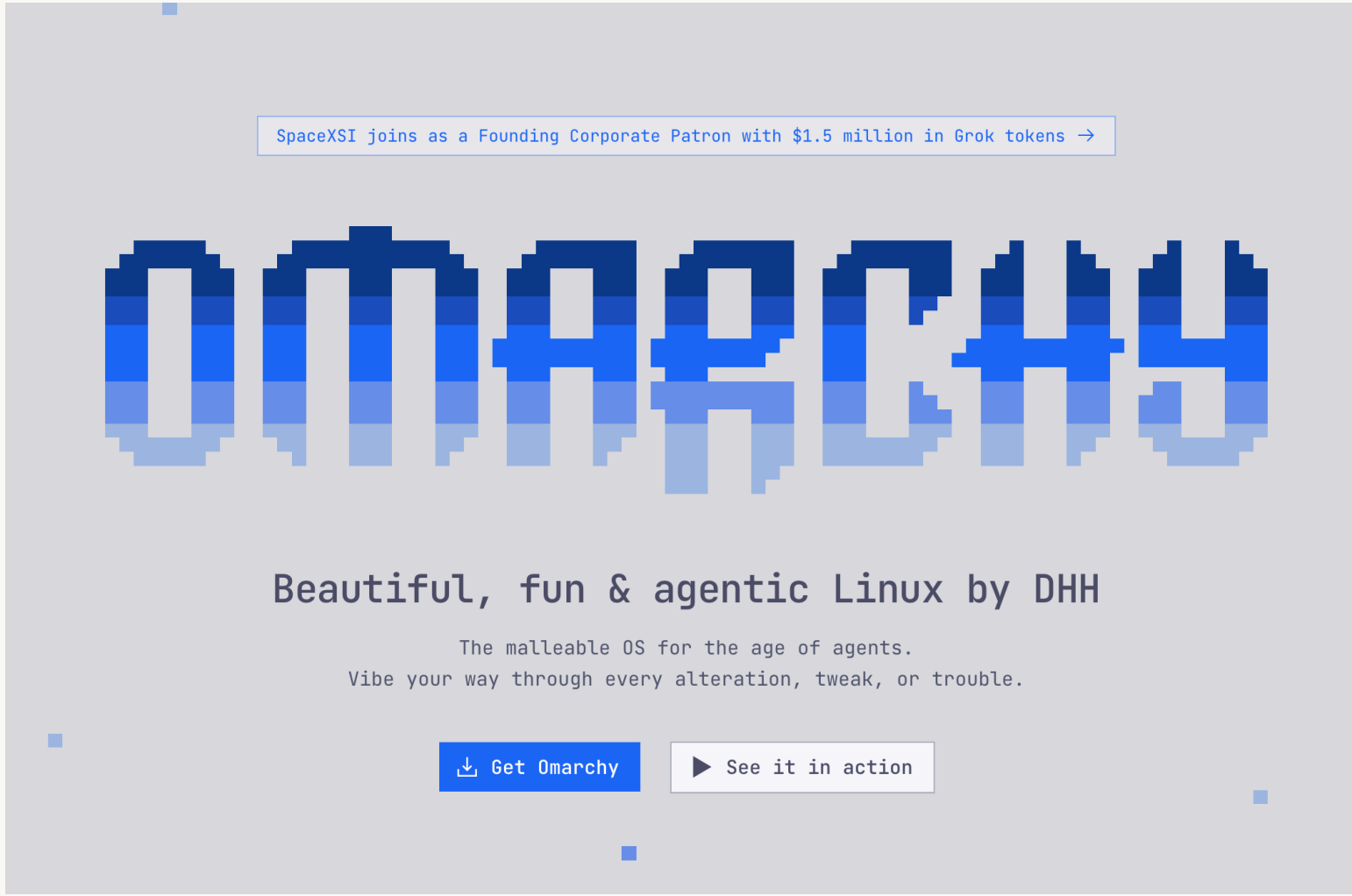
lemire.me/blog/2026/09/05/ai-programming-a-layered-model/



Emerging idea: cheap custom software

1. Build your own video software https://github.com/lemire/lemirecut
2. Your own election-night app https://lemire.me/montmorency2026/

Emerging idea: AI-first operating software



https://simdutf.github.io/simdutf/ · https://fastfloat.github.io/fast_float/

🇨🇦

TODO

- fast paced bug catching https://x.com/lemire/status/2096307923861336399?s=20
- add https://lemire.me/blog/2026/09/05/ai-programming-a-layered-model/
- verfiable vs non-verifiable, Cette notion de ce qui est vérifiable et de ce qui est difficilement vérifiable.
- more performant software (bar higher), more correct software
- historical reviewers
- explicit knowledge, genie in the bottle
- AI and credentialism, reputation system
- system understanding, animation, videos

who use these tools daily for software engineering yet say they hate “AI slop.”

- **Ninad Sachania (@NinadSachania)**: Gell-Mann Amnesia (the tendency to trust reporting/expertise in domains you do not know while recognizing its flaws in domains you do know—relevant to how people assess AI-generated code).
- **aa1337**: Demonstrate AI reading all your blog posts and using that knowledge to obtain large speedups on a widely used piece of code.
- **Ronan Tournier (@RonanTournier)** (reply to your thank-you post): The need for ever more coding examples to feed the models. He is impressed by current results but notes the scarcity for advanced SQL—there simply are not enough high-quality advanced examples online (SQL standing in for any domain whose sophisticated code is under-represented on the public internet). (He also corrected a typo: “feed” not “feet.”)

Practical takeaways for the talk

The recurring threads are:

1. Verification and judgment remain the bottleneck (testing quality, explaining why

"/Users/dlemire/Desktop/ipo.png"